



Banco de Dados Técnicas de Programação SQL

Marcel Oliveira

1



Programação para BD

- A maioria dos SGBDs têm uma interface interativa como a que vimos
 - Digitação de comandos
 - Carregamento de arquivo com comandos
 - Conveniente para
 - Criação do esquema
 - Criação de restrições
 - Consultas ad-hoc

2



Programação para BD

- Na prática, a maioria das interações são feitas através de programas “*cuidadosamente projetados e testados*”
 - Aplicações de BD
 - Interfaces Web
 - Servlets
 - PHP
 - Como acessar um BD a partir de um programa?

3



Programação para BD

- Assunto bastante vasto
- Várias técnicas e livros dedicados a cada uma delas
- Criação de novas técnicas acontecem constantemente

4



Nesta Aula

- Introdução básica a algumas técnicas de programação para BD
- Exemplo prático de duas delas

5



Programação de BD

- *Linguagem hospedeira*
 - Linguagem de programação (Java, COBOL, C++, ...)
- *Sub-linguagem de dados*
 - SQL
- Em casos especiais
 - *Linguagens de programação de BD*

6



Abordagens

- Comandos embutidos
 - Comandos de BD são embutidos em uma linguagem de programação de propósito geral
- Biblioteca de funções de BD
 - Disponível para a linguagem hospedeira para chamadas ao BD (API)
- Uma linguagem completamente nova
 - Minimiza o descompasso de impedância (*impedance mismatch*)

7



Abordagens

- Mais comuns
 - Comandos embutidos
 - Biblioteca de funções de BD
- Ideal para aplicações com interação intensiva com o BD
 - Uma linguagem completamente nova

8



Descompasso de impedância (*impedance mismatch*)

- Incompatibilidade entre o modelo da linguagem hospedeira e o modelo do BD
 - Tipos de dados
 - Ligações de dados
 - Resultados de consultas são *bags* de tuplas (sequência de valores de atributos)
 - Ligações de estrutura de dados
 - Uso de cursor para fazer o *loop*

9



Interação com o BD

1. Programa cliente abre uma conexão com o servidor de BD
2. Programa cliente submete consultas/atualizações para o BD
3. Quando o acesso ao BD não é mais necessário o programa cliente fecha a conexão

10



Comandos Embutidos

- A maioria dos comandos SQL podem ser embutidos em uma linguagem de programação
 - Definições de dados, restrições, consultas, atualizações, visões
- Distinguímos os comandos SQL dos comandos da linguagem hospedeira
 - Sintaxe varia com a linguagem
 - EXEC SQL ou EXEC SQL BEGIN
 - END-EXEC ou EXEC SQL END
- Variáveis compartilhadas
 - Geralmente prefixadas com (:) em SQL
- Pré-processador pode separar os comandos SQL embutidos do programa na linguagem hospedeira

11



Declaração de Variáveis em C

- Variáveis declaradas dentro de DECLARE são compartilhadas e podem aparecer (prefixadas com :) em declarações SQL

Usadas para comunicar erros/exceções entre o BD e o programa

```
0) int loop;  
1) EXEC SQL BEGIN DECLARE SECTION ;  
2) varchar dnome [16], pnome [16], unome [16], endereço [31] ;  
3) char ssn [10], datnasc [11], sexo [2], inicial [2] ;  
4) float salario, aumento;  
5) int dno, dnumero;  
6) int SQLCODE ; char SQLSTATE [6] ;  
7) EXEC SQL END DECLARE SECTION ;
```

12



Comandos SQL para Conexão com o BD

- Conexão (várias conexões são possíveis mas apenas uma pode estar ativa)

```
CONNECT TO server-name  
AS connection-name  
AUTHORIZATION user-account-info;
```

- Mudança de conexão ativa

```
SET CONNECTION connection-name;
```

- Desconexão

```
DISCONNECT connection-name;
```

13



SQL Embutido em C Exemplo

```
//Segmento de Programa E1:  
0) loop = 1 ;  
1) while (loop) {  
2)     prompt ("Entre com o Numero do Seguro Social: ", ssn) ;  
3)     EXEC SQL  
4)         select PNAME, MINICIAL, UNOME, ENDEREÇO, SALARIO  
5)         into :pname, :minicial, :unome, :endereco, :salario  
6)         from EMPREGADO where SSN = :ssn ;  
7)     if (SQLCODE == 0) printf (pname, minit, unome, endereco, salario)  
8)     else printf ("Numero do Seguro Social nao existe: ", ssn) ;  
9)     prompt("Mais Numeros de Seguro Social (entre 1 para Sim, 0 para Nao): ", loop) ;  
10) }
```

14



SQL Embutido em C Múltiplas Tuplas

- Precisamos de um cursor (*iterator*) para processar múltiplas tuplas
- Comando **FETCH** move o cursor para a próxima tuplas
- **CLOSE CURSOR** indica que o processamento da consulta foi completado

15



SQL Embutido em C Múltiplas Tuplas

```
//Segmento de Programa E2:
0) prompt ("Entre com o Nome do Departamento: ", dnome) ;
1) EXEC SQL
2)     select DNUMERO into :dnumero
3)     from DEPARTAMENTO where DNAME = :dnome ;
4) EXEC SQL DECLARE EMP CURSOR FOR
5)     select SSN, PNAME, MINICIAL, UNOME, SALARIO
6)     from EMPREGADO where DNO = :dnumero
7)     FOR UPDATE OF SALARIO ;
8) EXEC SQL OPEN EMP ;
9) EXEC SQL FETCH from EMP into :ssn, :pname, :minicial, :unome, :salario ;
10) while (SQLCODE == 0) {
11)     printf("Nome do empregado e:", pname, minit, unome)
12)     prompt("Entre com o aumento de salario: ", aumento) ;
13)     EXEC SQL
14)         update EMPREGADO
15)         set SALARIO = SALARIO + :raise
16)         where CURRENT OF EMP ;
17)     EXEC SQL FETCH from EMP into :ssn, :pname, :minicial, :unome, :salario ;
18) }
19) EXEC SQL CLOSE EMP ;
```

Não executa a consulta

Executa a consulta

Na tupla atual do cursor

16



SQL Dinâmica

- Na SQL embutida, cada nova consulta implica na alteração do código-fonte
- SQL Dinâmica
 - Compor e executar declarações SQL em tempo de execução (não previamente compiladas)
 - Um programa que aceita comandos SQL do teclado em tempo de execução
 - Uma operação de apontar e clicar traduz para certas consultas SQL
- Atualização dinâmica é relativamente simples, mas consulta dinâmica pode ser complexa
 - Tipos e números de atributos recuperados são desconhecidos em tempo de compilação

17



SQL Dinâmica

```
//Segmento de Programa E3:  
0) EXEC SQL BEGIN DECLARE SECTION ;  
1) varchar sqlatualizacadeia [256] ;  
2) EXEC SQL END DECLARE SECTION ;  
...  
3) prompt ("Entre com o Comando de Atualizacao: ", sqlatualizacadeia) ;  
4) EXEC SQL PREPARE sqlcomando FROM :sqlatualizacadeia ;  
5) EXEC SQL EXECUTE sqlcomando ;  
...
```

18



SQLJ

- Padrão para embutir SQL em Java
- O tradutor SQLJ transforma os comandos SQL em Java usando a interface JDBC
 - Executados via a interface *JDBC*

19



SQLJ

- Importando pacotes necessários
- Abrindo conexão
- Estabelecendo o contexto padrão

```
1) import java.sql.* ;
2) import java.io.*;
3) import sqlj.runtime.* ;
4) import sqlj.runtime.ref.* ;
5) import oracle.sqlj.runtime.* ;
...
6) DefaultContext cntxt =
7)   oracle.getConnection("<nome url>", "<nome usuario>", "<senha>", true) ;
8) DefaultContext.setDefaultContext(cntxt) ;
...
```

20



SQLJ Consultas

```
1) String dnome, ssn , pnome, fn, unome, ln, datanasc, endereco ;
2) Char sexo, minicial, mi ;
3) double salario, sal ;
4) Integer dno, dnumero ;

//Segmento de Programa J1:
1) ssn = readEntry("Entre com o Numero do Seguro Social: ") ;
2) try {
3)     #sql {select PNAME, MINICIAL, UNOME, ENDERECO, SALARIO
4)         into :pname, :minicial, :unome, :endereco, :salario
5)         from EMPREGADO where SSN = :ssn} ;
6) } catch (SQLException se) {
7)     System.out.println("Numero do Seguro Social Inexistente: " + ssn) ;
8)     Return ;
9) }
10) System.out.println(pnome + " " + minicial + " " + unome + " " + endereco + " " +
    salario)
```

21



SQLJ Diversas Tuplas

- Dois tipos de iteradores
 - Designado
 - Associado ao resultado de uma consulta por meio de uma lista de tipos e nomes dos atributos
 - Posicional
 - Lista somente os tipos dos atributos

22



SQLJ Iterator Designado

```
//Segmento de Programa J2A:
0)  dnome = readEntry ("Entre com o Nome do Departamento: ") ;
1)  try {
2)      #sql {select DNUMERO into :dnumero
3)          from DEPARTAMENTO where DNAME = :dnome} ;
4)  } catch (SQLException se) {
5)      System.out.println ("Departamento Inexistente: " + dnome) ;
6)      Return ;
7)  }
8)  System.out.println("Informacoes dos Empregados do Departamento: " + dnome) ;
9)  #sql iterator Emp(String ssn, String pname, String minicial, String unome,
double salario) ;
10) Emp e = null ;
11) #sql e = {select ssn, pname, minicial, unome, salario
12)         from EMPREGADO where DNO = :dnumero} ;
13) while (e.next()) {
14)     System.out.println (e.ssn + " " + e.pnome + " " + e.minicial + " " +
15)         e.unome + " " + e.salario) ;
16) } ;
16) e.close() ;
```

23



SQLJ Iterator Posicional

```
//Segmento de Programa J2B:
0)  dnome = readEntry ("Entre com o Nome do Departamento: ") ;
1)  try {
2)      #sql {select DNUMERO into :dnumero
3)          from DEPARTAMENTO where DNAME = :dnome} ;
4)  } catch (SQLException se) {
5)      System.out.println ("Departamento Inexistente: " + dnome) ;
6)      Return ;
7)  }
8)  System.out.println ("Informacoes dos Empregados do Departamento: " + dnome) ;
9)  #sql iterator Emppos (String, String, String, String, double) ;
10) Emppos e = null ;
11) #sql e = {select ssn, pname, minicial, unome, salario
12)         from EMPREGADO where DNO = :dnumero} ;
13) #sql {fetch :e into :ssn, :fn, :mi, :ln, :sal} ;
14) while (!e.endFetch()) {
15)     System.out.println (ssn + " " + fn + " " + mi + " " + ln + " " + sal) ;
16)     #sql {fetch :e into :ssn, :fn, :mi, :ln, :sal} ;
17) } ;
18) e.close() ;
```

24



Bibliotecas de Acesso JDBC

- Java Database Connectivity
- Biblioteca de funções de BD
 - Disponível para a linguagem hospedeira para chamadas ao BD (API)
- Um programa Java com funções JDBC pode acessar qualquer SGBD relacional que tem um driver JDBC
- JDBC permite que um programa se conecte a vários BDs (fontes de dados)

25



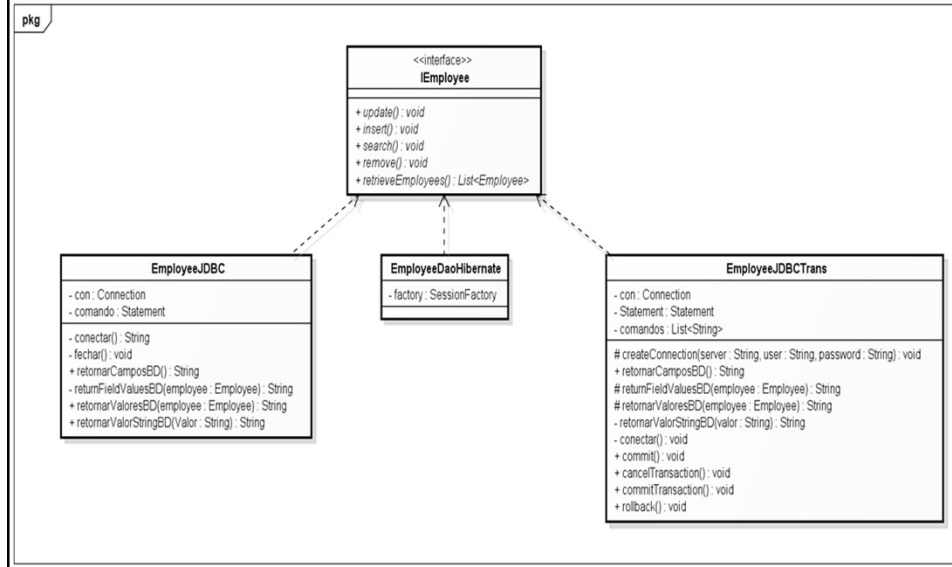
JDBC Passos do Acesso ao BD

1. Importar a biblioteca JDBC
(`java.sql.*`)
2. Definir variáveis apropriadas
3. Criar uma conexão (`Connection`)
4. Criar um comando SQL (`Statement`)
6. Identificar parâmetros do comando SQL (designados por ?)
7. Ligar parâmetros a variáveis do programa
8. Executar comando SQL via JDBC
(`executeQuery`)
9. Processar o resultado (`ResultSet`)
ResultSet é uma tabela bi-dimensional

26



JDBC Exemplo



27



JDBC Exemplo

- Código utilizando apenas JDBC

28



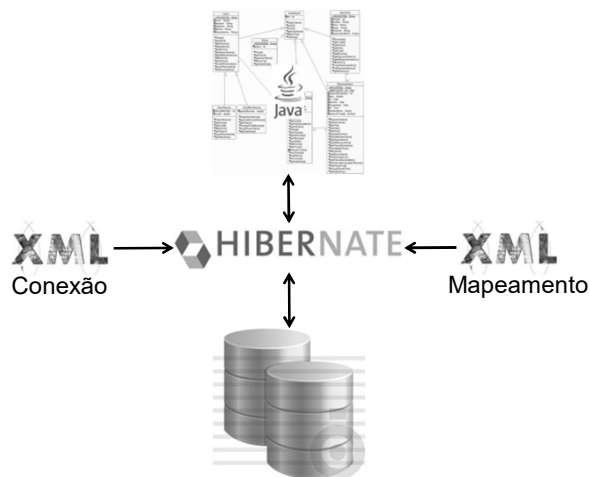
Hibernate

- Facilita o mapeamento OO-Relacional
- Transforma (vice-versa)
 - Classes Java em tabelas de dados
 - Tipos de dados
- Gera chamadas SQL e libera o desenvolvedor do trabalho manual de conversão dos dados resultantes
 - Portabilidade entre SGBDs

29



Hibernate



30



Hibernate

- Passos para uso
 - Baixar Hibernate (www.hibernate.org) e importar os .jar da pasta hibernate\lib para a lib do seu projeto
 - Criar a tabela no seu banco de dados onde os objetos vão persistir;
 - Criar a classe cujo estado vai ser persistido;
 - Criar um XML, que relaciona as propriedades do objeto aos campos na tabela

31



Hibernate

- Passos para uso
 - Criar arquivos contendo as propriedades para que o *Hibernate* se conecte ao banco de dados
 - Criar a classe DAO que vai persistir seu objeto;

32



Hibernate

- Exemplo do projeto
 - Criação da classe da regra de negócio (Employee)
 - Criar um XML, que relaciona as propriedades do objeto aos campos na tabela (Employee.hbm.xml)
 - Criar arquivos contendo as propriedades para que o *Hibernate* se conecte ao banco de dados (hibernate.properties)
 - log4j.properties
 - Criar a classe DAO que vai persistir seu objeto (EmployeeDaoHibernate.java)

33



Programação BD com Chamadas de Funções

- SQL embutido provê programação BD estática
- API: programação BD dinâmica com uma biblioteca de funções
 - Vantagem: não precisa de pré-processamento (mais flexível)
 - Desvantagem: checagem do SQL é feita em tempo de execução

34



SQL/CLI

- *Call Level Interface*
- Parte do padrão SQL
- Provê fácil acesso a vários BDs dentro do mesmo programa
- Algumas bibliotecas (p. ex., `sqlcli.h` para C) têm que estar instaladas no servidor e disponíveis
- Comandos SQL são criados dinamicamente e passados como parâmetros `String` nas chamadas

35



SQL/CLI

- 4 Componentes
 - *Registro de ambiente:*
 - Mantém as informações de controle sobre as conexões de BD e registra as informações do ambiente
 - *Registro de Conexão:*
 - Cria informações de controle de conexão de um BD em particular
 - *Registro de Declaração:*
 - Mantém o controle das informações necessárias para um comando SQL
 - *Registro de Descrição:*
 - Mantém informações sobre as tuplas

36



SQL/CLI Passos

1. Carregar bibliotecas SQL/CLI
2. Declarar variáveis de registro para os componentes descritos (SQLHSTMT, SQLHDBC, SQLHENV, SQLHDEC)
3. Configurar um registro de ambiente usando SQLAllocHandle
4. Configurar um registro de conexão usando SQLAllocHandle
5. Configurar um registro de declaração usando SQLAllocHandle
6. Preparar um comando usando a função SQL/CLI SQLPrepare
7. Ligar parâmetros a variáveis do programa
8. Executar comando SQL usando SQLExecute
9. Ligar coluna da consulta a variáveis do programa usando SQLBindCol
10. Usar SQLFetch para recuperar os valores das colunas em variáveis de programa

37



SQL/CLI Exemplo

```
//Programa CLI1:
0)  #include sqlcli.h ;
1)  void printSal() {
2)    SQLHSTMT stmt1 ;
3)    SQLHDBC con1 ;
4)    SQLHENV env1 ;
5)    SQLRETURN ret1, ret2, ret3, ret4 ;
6)    ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &env1) ;
7)    if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, env1, &con1) else exit ;
8)    if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz", SQL_NTS)
    else exit ;
9)    if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &stmt1) else exit ;
10)   SQLPrepare (stmt1, "select UNOME, SALARIO from EMPREGADO where SSN = ?", SQL_NTS) ;
11)   prompt("Entre com o numero do seguro social: ", ssn) ;
12)   SQLBindParameter (stmt1, 1, SQL_CHAR, &ssn, 9, &fetchlen1) ;
13)   ret1 = SQLExecute(stmt1) ;
14)   if (!ret1) {
15)     SQLBindCol (stmt1, 1, SQL_CHAR, &unome, 15, &fetchlen1) ;
16)     SQLBindCol (stmt1, 2, SQL_FLOAT, &salario, 4, &fetchlen2) ;
17)     ret2 = SQLFetch (stmt1) ;
18)     if (!ret2) printf(ssn, unome, salario)
19)       else printf ("Numero do Seguro Social Inexistente: ", ssn) ;
20)   }
21) }
```

38



Procedimentos Armazenados

- Funções e procedimentos (módulos) são armazenados localmente e executados pelo SGBD
 - Ao contrário da execução no cliente
- Vantagens
 - Se o procedimento é necessário para muitas aplicações, ele pode ser invocado por quaisquer uma delas (evitando duplicações)
 - Execução pelo servidor reduz custos de comunicação
 - Aumenta o poder de definição de visões e gatilhos
- Desvantagens:
 - Problemas de portabilidade
 - Todo SGBD tem sua própria sintaxe
 - Alguns SGBDs permitem a definição de procedimentos em linguagens de programação de propósito geral

39



Construtores

- Um procedimento armazenado

```
CREATE PROCEDURE
    procedure-name (params)
    local-declarations
    procedure-body;
```
- Uma função armazenada

```
CREATE FUNCTION
    fun-name (params)
    RETURNS return-type
    local-declarations
    function-body;
```
- Chamando um procedimento ou função

```
CALL procedure-name/fun-name (arguments);
```

40



Módulos Armazenados de Modo Persistente

- SQL/PSM:
 - Parte do padrão SQL para escrever módulos armazenados persistentes
- SQL + procedimentos/funções armazenadas + construções de programação adicional
 - Comandos de ramificações e loops
 - Aumenta o poder de SQL

41



Módulos Armazenados de Modo Persistente

```
CREATE FUNCTION DEPT_SIZE
  (IN deptno INTEGER)
  RETURNS VARCHAR[7]

DECLARE TOT_EMPS INTEGER;

SELECT COUNT (*) INTO TOT_EMPS
  FROM SELECT EMPLOYEE WHERE DNO = deptno;
IF TOT_EMPS > 100 THEN RETURN "HUGE"
  ELSEIF TOT_EMPS > 50 THEN RETURN "LARGE"
  ELSEIF TOT_EMPS > 30 THEN RETURN "MEDIUM"
  ELSE RETURN "SMALL"
ENDIF;
```

42



SQL Injection

- Ameaça à segurança e integridade de sistemas que interagem com bases de dados via SQL
- Uso de *strings* maliciosas



```
String sql = "DELETE FROM EMPLOYEE  
WHERE ssn=" + ssn;
```

43



SQL Injection



```
String sql = "DELETE FROM EMPLOYEE  
WHERE ssn=" + ssn;
```



```
String sql = "DELETE FROM EMPLOYEE  
WHERE ssn=111111100 OR TRUE";
```

- Exemplo
 - EmployeeJDBC.unsafeRemove(...)

44



SQL Injection

- Principais soluções
 - **PreparedStatement:** principal mecanismo de defesa.
 - **Whitelist Validation:** para partes da consulta que não podem ser parametrizadas.
 - **Princípio do Menor Privilégio (Least Privilege):** reduzir o impacto caso uma injeção consiga ocorrer.

45



SQL Injection

- PreparedStatement
 - **Proteção contra SQL Injection** através da separação entre comando SQL e parâmetros.
 - **Melhor desempenho** em consultas executadas repetidamente, permitindo reutilização do plano de execução.
 - **Código mais legível e fácil de manter** do que consultas montadas por concatenação. **Maior portabilidade** entre diferentes bancos de dados e drivers JDBC.
- Exemplo
 - `EmployeeJDBCPreparedStatement`

46



SQL Injection

- PreparedStatement protege apenas os parâmetros da consulta, mas não protege
 - Nome da tabela ou Nome da coluna
 - ORDER BY e ASC/DESC

```
String sql =  
    "SELECT *  
    FROM employee  
    ORDER BY " + str;
```



```
"fname; DROP TABLE employee"
```

47



SQL Injection

- *Whitelist validation*
 - técnica de segurança que aceita apenas valores previamente autorizados, bloqueando qualquer entrada fora da lista permitida.

```
Set<String> validFields =  
    Set.of("fname", "lname");  
  
if (! validFields.contains(str)) {  
    throw new IllegalArgumentException("...");  
}  
  
String sql =  
    "SELECT * FROM employee ORDER BY " + str;
```

48



SQL Injection

- Princípio do Menor Privilégio (*Least Privilege*)
 - Conceda a cada usuário apenas as permissões estritamente necessárias para executar suas funções.
 - Usuário da aplicação apenas com os acessos necessários

```
GRANT SELECT, INSERT, UPDATE  
ON employee  
TO app_user;
```

49



SQL Injection

- **Resumo**
 - **PreparedStatement**: separam SQL e parâmetros e previne a maioria dos ataques.
 - **Whitelist Validation**: restringe elementos dinâmicos da consulta e protege a estrutura da consulta.
 - **Menor Privilégio**: limita o impacto de uma eventual exploração e reduz os danos caso algo passe pelas defesas.
- Esse trio cobre praticamente todos os principais riscos sobre SQL Injection.

50



SQL Injection

- Outras técnicas
 - Sanitização de entrada: Remove ou trata caracteres considerados problemáticos antes do processamento.
 - Escape de caracteres: Adiciona caracteres de escape para impedir que aspas e outros símbolos sejam interpretados como parte do SQL.

51



SQL Injection

- Outras técnicas
 - WAF (*Web Application Firewall*): Ferramenta que monitora e filtra requisições suspeitas antes que cheguem à aplicação.
 - ORM (Hibernate/JPA): Ferramentas como Hibernate geram consultas parametrizadas automaticamente na maioria dos casos.

52



SQL Injection

- Outras técnicas
 - Procedimentos Armazenados: Rotinas armazenadas no banco de dados que encapsulam consultas e regras de negócio.
 - Monitoramento e auditoria: Registro e análise de consultas, erros e atividades suspeitas.

53



Dados Criptografados

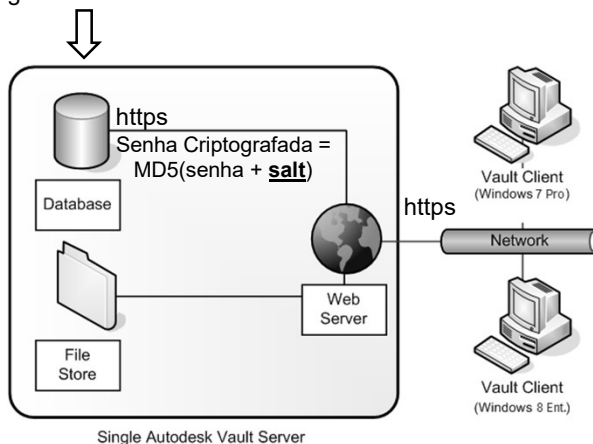
- Como armazenar informações sigilosas (senhas) no BD?
- Como enviar/receber estas informações?

54



Dados Criptografados

Restringir acesso a IP do servidor



55

Dados Criptografados

- Escolha do “sal”
 - Um por usuário
 - Um para todos guardado localmente
 - Combinação

56



57



58



Resumo

- Um BD pode ser acessado de modo interativo
- Geralmente, um BD é acessado via programas de aplicação
- Vários métodos de programação de BD
 - SQL embutido
 - SQL dinâmico
 - Procedimentos e funções armazenadas

59



Resumo

- SQL Injection
- Dados Criptografados

60



Leitura

- Capítulo 13

61



62